

Railway PVC System - Project Instructions

Zone-Based Steel City Pricing (March 2025)

Architecture

- **Mapping file:** `lib/zone-steel-city-mapping.ts` maps 18 Indian Railway zones to 4 JPC steel cities (Delhi, Mumbai, Chennai, Kolkata)
- **Key exports:** `getRailwayZoneOptions()`, `getSteelCityForZone()`, `getSteelIndexNamesForZone()`, `getZoneDisplayName()`
- **Chennai is default:** Unsuffix index names (`Steel TMT Bars` etc.) are Chennai rates. For Chennai zones, `getSteelIndexNamesForZone()` returns unsuffixed names (backward compatible)
- **City-specific indices in DB:** `Steel TMT Bars`, `Steel TMT Bars - Delhi`, `Steel TMT Bars - Mumbai`, `Steel TMT Bars - Chennai`, `Steel TMT Bars - Kolkata` (same pattern for Angle/Channel, Plates, Other Sections)
- **Legacy handling:** `getSteelCityForZone()` handles both zone codes (SR, CR) AND legacy city names (Chennai, Mumbai)

Files Modified for Zone-Aware Steel Pricing

- **Frontend dropdowns:** `bills/new/page.tsx`, `bills/edit/[id]/page.tsx`, `bills/bulk-new/page.tsx` — all use `getRailwayZoneOptions()`
- **Bill CREATE:** `api/bills/route.ts` — uses `getSteelIndexNamesForZone(zone)`
- **Bill UPDATE:** `api/bills/[id]/route.ts` — uses `getSteelIndexNamesForZone(zone)`
- **Bill RECALCULATE:** `api/bills/[id]/recalculate/route.ts` — uses `getSteelIndexNamesForZone(bill.zone)`
- **Bulk CREATE:** `api/bills/bulk-create/route.ts` — uses `getSteelIndexNamesForZone(billInput.zone)`, saves zone to DB
- **Calculation steps:** `api/bills/[id]/calculation-steps/route.ts` — uses `getSteelIndexNamesForZone(bill.zone)`
- **Diagnostic:** `api/bills/diagnostic/route.ts` — uses `getSteelIndexNamesForZone(billZone)`
- **External bills:** `api/external/bills/route.ts` — uses `getSteelIndexNamesForZone(zone)`
- **PDF report:** `api/bills/[id]/pdf-report/route.ts` — uses `getSteelIndexNamesForZone(bill.zone)`
- **Bulk PDF report:** `api/bills/bulk-pdf-report/route.ts` — per-bill zone-aware indices via `getIndicesNamesForBill(bill.zone)`
- **Abstract report:** `api/reports/abstract/route.ts` — per-bill zone-aware indices
- **Extension compliance:** `lib/extension-compliance.ts` — steel lookups use prefix matching for city-suffixed names
- **PVC calculations:** `lib/pvc-calculations.ts` — `findSteelIndex()` helper does exact match then prefix match

Important Technical Notes

- `findSteelIndex()` in `pvc-calculations.ts` is a private helper that checks exact match first, then `baseName + ' - '` prefix
- Extension compliance uses similar inline prefix matching for `cappedIndices` and `indexL_Values`

- Bill model has `zone` field (String?, on `bills` table)
- User profile has separate `railwayZone` field (for railway officials)

Zone-Based Fuel City Pricing (March 2025)

Architecture

- **Same mapping file:** `lib/zone-steel-city-mapping.ts` — added `getFuelIndexNameForBill(zoneCode, fuelPriceType)` and `getFuelIndexNamesForBill(zoneCode, fuelPriceType)`
- **Key difference from steel:** Unsuffixed `'MPNG Fuel'` = 4-city average (NOT Chennai). City-specific ALWAYS uses suffix, even Chennai: `'MPNG Fuel - Chennai'`
- **Per-bill selection:** Bill model has `fuelPriceType` field (String?, default `'four_city_avg'`). Values: `'four_city_avg'` or `'zone_city'`
- **City fuel indices in DB:** `'MPNG Fuel'` (4-city avg), `'MPNG Fuel - Delhi'`, `'MPNG Fuel - Mumbai'`, `'MPNG Fuel - Chennai'`, `'MPNG Fuel - Kolkata'`

Fuel Sync

- **Route:** `api/indices/fuel-prices/sync/route.ts` — generates both 4-city average AND per-city monthly indices
- **Auto-creates indices:** `ensureCityFuelIndicesExist()` creates `PriceIndex` records on first sync
- **Source:** Uses `DailyFuelPrice` model which has per-city columns (delhi, mumbai, chennai, kolkata)

Files Modified for Fuel Pricing

- Same bill routes as steel (all 10 non-backup routes) + frontend forms
- **PVC calculations:** `findFuelIndex(indexMap)` — mirrors `findSteelIndex()`, exact match then prefix match
- **Extension compliance:** 3 fuel lookups updated with prefix matching fallback
- **PDF report:** Aliasing — city-specific fuel values also stored under `'MPNG Fuel'` key for backward-compatible formula display

Important Notes

- User must run fuel sync after deploy to populate city-specific monthly indices
- Zone→city mapping is shared between steel and fuel (same `ZONE_STEEL_CITY_MAP`)

Telegram Bot Integration (March 2025)

Architecture

- **Webhook endpoint:** `api/telegram/webhook/route.ts` — receives Telegram updates (messages + callback queries)
- **Webhook setup:** `api/telegram/setup-webhook/route.ts` — GET endpoint to register webhook URL with Telegram
- **Telegram API utilities:** `lib/telegram-api.ts` — `sendMessage`, `sendDocument`, `setWebhook`, keyboard helpers
- **Conversation management:** `lib/telegram-conversation.ts` — `getOrCreate`, `update`, `reset`, `linkToUser` via phone
- **Message handler:** `lib/telegram-message-handler.ts` — full bill creation flow (mirrors WhatsApp)
- **DB model:** `TelegramConversation` in `schema.prisma` with `chatId` (unique string) as key

Flow

Same as WhatsApp: phone linking → contract selection → bill no → amount → date → classification → cement → steel → steel types → confirm → create bill

Key Differences from WhatsApp

- Uses `chatId` instead of phone number for conversation tracking
- User must manually enter registered phone number to link account (AWAITING_PHONE step)
- Uses HTML parse mode for formatting (`<b>` , `<i>`) instead of WhatsApp markdown
- Uses inline keyboard buttons for selections (callback_query)
- Bot token: stored as `TELEGRAM_BOT_TOKEN` in `.env`
- Middleware exclusion: `api/telegram` excluded from auth middleware

Index Status Fix (March 2025)

- Core indices = 10 (excludes city-specific variants like `/ - (Delhi|Mumbai|Chennai|Kolkata)$/`)
- Files fixed: `lib/index-status.ts` , `api/indices/latest-available/route.ts` , `api/indices/validate-date/route.ts` , `api/indices/available-final-months/route.ts` , `api/indices/available-date-range/route.ts`

Completed Indices Only Display (March 31, 2026)

Overview

- **Refactored** `IndicesAvailabilityIndicator` **component** - now shows ONLY completed indices
- Removes all incomplete current month information completely
- Shows latest completed month with clean, minimal display
- Simplest possible view focused on available data

Implementation

File: `components/indices-availability-indicator.tsx` (major refactor - ~200 lines removed)

Display Format

✓ Completed up to: January 2026 (Provisional) • 46 complete months

OR (if no completed data):

⚠ No completed indices available yet

Features

- **Completed Only:** Shows ONLY months where ALL 10 core price indices are available
- **No Incomplete Data:** Removed entirely - never displays incomplete month information
- **Status Badge:** Shows “(Provisional)” only if the month has any provisional indices
- **Month Count:** Displays total number of completed months at a glance
- **Visual Design:**
 - Green card with pulsing checkmark for completed state
 - Amber card with alert icon for no data state
 - Gradient backgrounds and animations

Changes Made

1. **Removed 199 lines** of code that displayed incomplete/latest month status
2. **Simplified component** to single responsibility: display completed months only
3. **Removed prop:** `showOnlyCompleted` - no longer needed (always shows completed)
4. **Cleaned imports:** Removed unused `Link`, `XCircle`, `Calendar`, `Database` icons

PDF Component Document Deduplication Fix (March 31, 2025)

Issue

- When bills used multiple steel types (TMT, Angle/Channel, Plates, Other Sections) or when multiple component types referenced the same JPC document, the same document was embedded multiple times in PDF reports.
- Example: A bill with TMT, Angle/Channel, and Plates would embed the JPC document 3 times if all referenced the same physical file.

Root Cause

- If the same physical JPC document was uploaded/stored with different component type labels, or referenced under different component types, it would be embedded once per component type.

Solution

- Added deduplication by `cloudStoragePath` in two functions:
- `embedComponentIndices()` : Single bill PDF reports
- `embedComponentIndicesRange()` : Bulk/multi-bill PDF reports
- Creates a Map keyed by cloud storage path to ensure each unique physical document file is embedded only once
- Added informative logs: "X total documents, Y unique by file (deduped by cloud storage path)"

Files Modified

- `lib/pdf/utils/labour-index-embedder.ts` :
- Both functions now deduplicate by cloud storage path before embedding
- Simplified sorting logic by removing nested component type grouping
- Direct iteration over deduplicated documents